CO4_AT2

SIMATS ENGINEERING
ASSESSMENT TOOLS

COURSE NAME: Theory of Computation COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO4: *Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5 Total Marks:25
Duration : 1 Hour

Industry Problem

Code of Conduct:

Shaik Hafeeza Tarannum

I 192372044 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Hafeeza

SIMATS ENGINEERING
ASSESSMENT TOOLS

1.Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module sh ould accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$L = \{w \mid w \in (0+1)* \text{ and } n_0(w) = n_1(w)\}$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice than that of a's.

L={a^n b^2n | n≥1}

4. Develop a **C-based computational module** to determine the **ε-closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ε-transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- **S → 0 A 1**
- **A → 0 A | 1 A | ε**

(1) DFA for strings starting and Ending with a.

Language:

$L = \{w \in \{a,b\}^* \mid w$ starts with a and Ends with a$\}$

DFA States:

- q_0 - Intial state
- q_1 - starting string with a and currently ends with a (Final state)
- q_2 - string start with a but currently ends with b.
- q_d - Dead state

Transition-Table

State	a	b
→q_0	q_1	q_d
*q_1	q_1	q_2
q_2	q_1	q_2
q_d	q_d	q_d

C program:

```
#include <stdio.h>
#include <string.h>
int main(){
    char str[100];
    int i, state=0;
    printf("Enter a string:");
    scanf("%s", str);
    for(int i=0; str[i]!='\0'; i++){
        char ch = str[i];
        switch(state){
            case 0:
                if(ch=='a')
                state=1
                else
                state=3
```

```
        break;
    case 1:
        if (ch == 'a')
            state = 1
        else if (ch == 'b')
            state = 2
        else
            state = 3;
        break;
    }
    if (state == 1)
        printf("Accepted\n");
    else
        printf("Rejected\n");
    return 0;
}
```

2. pushdown Automaton for Equal Number of 0s and 1s.

The question appears to refer to the language.

$L = \{w \in \{0,1\}^* \mid n_0(w) = n_1(w)\}$

where

- $n_0(w)$ = number of 0s in w
- $n_1(w)$ = number of 1s in w

PDA Idea:

The PDA uses the stack to cancel opposite symbols.

- when 0 is read, push 0 if the top is z_0 or 0; otherwise pop 1.
- when 1 is read, push 1 if the top is z_0 or 1; otherwise pop 0
- Accept when the input is finished and only as remains.

Formal pDA:

$$M = (\{q\}, \{0,1\}, \{0,1,z_0\}, \delta, q, z_0, F)$$

The PDA accepts when the stack returns to the initial stack symbol z_0, indicating that the number of 0s and 1s is equal.

Ex:
- 01 → Accepted
- 10 → Accepted
- 0011 → Accepted
- 0101 → Accepted
- 001 → Rejected

3. PDA for $L = a^n b^{2n} \mid n \geq 1$

Language

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

For every one a, there must be Exactly two b's

PDA Strategy:

1. For every a, push two stack symbols X.
2. For every b, pop one X.
3. Accept when all input is processed and the stack reaches the bottom symbol.

Transitions:

For each a:

$$\delta(q_0, a, Z_0) = \{q_0, XXZ_0\}$$

$$\delta(q_0, a, X) = (q_0, XXX)$$

When b starts:

$$\delta(q_0, b, X) = (q_1, \varepsilon)$$

For remains bs:

$$\delta(q_1, b, X) = (q_1, \varepsilon)$$

Accept when stack reaches Z_0:

$$\delta(q_1, \varepsilon, Z_0) = (q_f, Z_0)$$

Example:

For the string aabbx Rejected because:

2a ⇒ 4b are required

For string aabbbb: 2a ⇒ 4b.

∴ aabbbb → Accepted

4. a program to Find Eclosure of All states in anNFA

```
#include<stdio.h>
#define MAX 20
int n;
int episilon[MAX][MAX];
int visited[MAX];
void episilonclosure(int state){
    int i;
    visited[state]=1;
    printf("q%d",state);
    for(i=0;i<n;i++){
        if(episilon[state][i]==1&&!visited[i]){
            episilonclosure[i];
        }
    }
}
int main(){
int i,j,state;
printf("Enter No of states:");
scanf("%d",&n);
printf("Enter Episilon transition matrix:\n");
scanf("%d",&n);
for(i=0;i<n;i++){
        for(j=0;j<n;j++){
            scanf("%d",&episilon[i][j]);
        }
    }
    for(state=0;state<n;state++){
        for(j=0;i<n;j++)
            visitied[i]=0;
    printf("\nEpisilon closure of q%d={",state);
        episilonclosure(state);
    printf("}");
}
return 0;
}
```

If:

$q_0 \to q_1 \to q_2$

Through ε-transition then:

$\epsilon closure(q_0) = \{q_0, q_1, q_2\}$

C program for CFG.

The given Grammar is,

S → 0A1

A → 0A | 1A | ε

The grammar generates all binary strings that:

- Start with 0
- End with 1
- Have any combination of 0s and 1s in between.

C program:

```
#include <stdio.h>
#include <string.h>
int main(){
    char str[100];
    int len, valid=1, i;
    printf("Enter a binary string:");
    scanf("%s", str);
    len = strlen(str);
    if(len<2){
        valid=0;
    }
    if(str[0]!='0'){
        valid=0;
    }
    if(str[len-1]!='1'){
        valid=0;
    }
    for(int i=0; i<len; i++){
```

```
if (str[i] != '0' && str[i] != '1') {
        valid = 0;
        break;
    }
}
if (valid)
    printf("string is Accepted\n");
else
    printf("string is Rejected\n");
return 0;
}
```

examples

Input	Result
01	Accepted
001	Accepted
011	Accepted
01010	Accepted
10	Rejected
10	Rejected

SIMATS ENGINEERING
ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand